MoCHII User Guide

Abstract

How to build and run MoCHII (MOnte Carlo for H II regions): requirements, the complete input-parameter reference, output-file formats, worked examples keyed to the regression tests, and the runbook for adding an ion. The physics and validation are documented separately in `docs/MoCHII_physics.pdf`; the atomic-data fitting pipeline in `docs/MoCHII_fitting.pdf`.

Contents

1. Requirements and build **2**

2. Input parameters **2**

2.1 Photons and RNG . . . 2
2.2 Grid and dust density . . . 3
2.3 Ionizing/FUV band and the source . . . 5
2.4 Plane-parallel slab illumination . . . 7
2.5 Spectrum-file formats . . . 8
2.6 Initial state and iteration . . . 9
2.7 Convergence of the gas iteration . . . 10
2.8 Thermal balance and metals . . . 11
2.9 I-front re-refinement . . . 14
2.10 Dust in the band and dust emission . . . 15
2.11 Peel-off imaging . . . 17
2.12 Output . . . 18

3. Output files **18**

3.1 `<base>_rates.h5` . . . 18
3.2 Line and emissivity outputs . . . 18
3.3 Spectral and image outputs . . . 22
3.4 Reading outputs in Python . . . 22
3.5 Slices and cutouts . . . 23
3.6 He I 2^3S metastable and 10833 Å absorption . . . 24
3.7 Building AMR grids . . . 24

4. Worked examples (the regression tests) **25**

5. Adding an ion **26**

6. Practical notes **26**

1. Requirements and build

- Intel oneAPI MPI Fortran (`mpiifort`/`mpiifx`; GNU `mpif90` supported by the Makefile), HDF5 (default prefix `/data/opt/hdf5_intel`; override with `make HDF5_PREFIX=...`), CFITSIO.
- The SEDust dust-emission library, self-contained under `SEDust/` in this tree. Build it once with `cd SEDust/sed && ./build_lib.sh` (→ `SEDust/sed/lib/libsedust.a` and its `.mod` files, which the Makefile links). The SEDust *data* directory is read at run time and resolved automatically relative to the executable (Section 2.10).
- Python 3 with numpy/scipy (h5py, astropy, matplotlib, PyNeb for the test and pipeline scripts).

Build:

```
cd MoCHII
(cd SEDust/sed && ./build_lib.sh)   # on a fresh checkout
make            # -> MoCHII.x   (always a full clean rebuild)
make DEBUG=1    # bounds/traceback build
```

The SEDust library only needs the one-time `build_lib.sh` step; afterward `make` links the in-tree SEDust library. The Makefile intentionally performs a full rebuild every time (the MoCafe policy): there is no inter-module dependency tracking, and stale `.mod` files after editing `define.f90` otherwise cause runtime namelist errors.
Run:

```
mpirun -np 8 ./MoCHII.x input.in
```

MoCHII is MPI-only; the octree and all leaf arrays live in MPI-3 shared memory (one copy on each node).

2. Input parameters

All input is one `&parameters` namelist assigning fields of `par`. Parameters not listed keep the defaults shown. The switches compose: `gas_niter = 0` gives a single rates-only pass; `gas_niter > 0` iterates the ionization; `solve_te` adds the thermal balance; `use_metals` the trace-metal registry; `diffuse_field` the explicit diffuse field (forces case A); `refine_front` the I-front re-refinement; `ion_add_dust` the dust coupling.

2.1 Photons and RNG

parameter	default	meaning
`no_photons`	1e5	source packets in each iteration
`no_print`	1e7	progress-print stride
`iseed`	0	RNG seed (0 = from clock/pid)

parameter	default	meaning
`launch_sequence`	'random'	quasi-random launch: `'sobol'` = Owen-scrambled Sobol coordinates for the stellar (and diffuse) packet launch, indexed by the global photon number (MPI-count-independent launch set); transport/scattering stay pseudo-random; prefer `nphotons` = 2^m. `'random'` is the Mersenne Twister default
`qmc_seed`	12345	scramble key (`launch_sequence='sobol'`); fixed across the gas iterations; change it for independent replicates (their scatter is the RQMC error estimate)
`use_master_slave`	F	how the photons of one transport pass are split over the MPI ranks. F = static cyclic: rank r takes indices r+1, r+1+$n_{\rm proc}$, …; T = rank 0 becomes the master, serving batches of `num_send_at_once` consecutive indices to whichever rank asks next; **the master transports no photons**, so $n_{\rm proc}-1$ ranks do the work. Applies to the stellar loop, the diffuse LyC loop and (through the same routine) the peel-off imaging pass; the dust-emission infrared transport and the SEDust solve distribute leaves rather than photons and are unaffected. Worth ∼20% of the ionizing-band pass at 70 ranks, and it cuts the run-to-run spread of that pass from 28% to 1.5%. Under `launch_sequence='random'` each rank draws from its own stream, so T also selects the realization — statistically equivalent, not numerically comparable; with `'sobol'` the two settings transport the same packet set and differ only by summation order
`num_send_at_once`	10000	photon indices in one served batch (`use_master_slave=T` only)

2.2 Grid and dust density

parameter	default	meaning
`grid_type`	'car'	`'amr'` (octree) or `'car'` (single-level Cartesian, leaf list in raster order; no tree and no geometry arrays in memory — cell centers and half-widths are computed analytically from the raster index; `refine_front` unavailable). `'uniform'` is accepted as an alias for `'car'`

parameter	default	meaning
`car_walk`	'dda'	car-grid traversal: `'dda'` (default) = dedicated incremental Amanatides–Woo walks ($t_{\max}/t_{\Delta}$ per axis, comparisons and additions only — the fastest car walk); `'amr'` = the octree walks with integer index arithmetic replacing the neighbor table, a verification mode that is bit-identical to the single-level octree (`grid_type = 'amr'`)
`amr_file`	''	generic AMR leaf file (FITS/HDF5/text): columns x, y, z, level, nH [, metallicity, xHI, ndust]. If empty, a `'car'` grid is built from the namelist instead (`'amr'` requires the file)
`nx, ny, nz`	1, 1, 11	`'car'`-from-namelist cell counts (cubic cells required: $2x_{\max}/n_x = 2y_{\max}/n_y = 2z_{\max}/n_z$)
`xmax, ymax, zmax`	1, 1, 1	`'car'`-from-namelist box half-extents [`distance_unit`]; box is $[-x_{\max}, x_{\max}]$ etc.
`nH_const`	−1	if ≥ 0, override every leaf's $n_{\rm H}$ with this uniform value [cm^{-3}] (both `'car'` and `'amr'`); < 0 keeps the file/column density
`density_file`	''	read the leaf $n_{\rm H}$ from a uniform 3D density cube onto a `'car'` grid (FITS 3D image, or the MoCHII HDF5 image format; NAXIS1/2/3 = $n_x/n_y/n_z$, values $n_{\rm H}$ [cm^{-3}]). $n_x/n_y/n_z$ come from the file, the box is `xmax/ymax/zmax`; an alternative to `amr_file`. `reduce_factor`> 1 downsamples; a `rmax/rmin` cut can mask the cube to a sphere
`rmax, rmin`	−999, 0	spherical density cut on leaf centers (both grids): $n_{\rm H} = 0$ for $r > r_{\max}$ (`rmax`> 0) and for $r < r_{\min}$ (`rmin`> 0; a shell); radii in `distance_unit`
`distance_unit`	'kpc'	'kpc'/'pc'/'au'/'' (code units)

parameter	default	meaning
`dust_density_law`	'global_dgr'	leaf grey opacity: `'none'`, `'global_dgr'` ($n_{\rm H} c_{\rm ext}$·DGR), `'from_file'` (ndust column), `'laursen09'` (static xHI column), `'laursen09_live'` (computed $x_{\rm HI}$, refreshed each iteration)

parameter	default	meaning
`cext_dust`	−1	reference extinction per H [cm^2] at `lambda_ref`, which every leaf's grey opacity is n_{dust} times. It is a property of the grains the run carries, so NEGATIVE (the default) takes it from them — from `dust_model`, or from the table `ion_dust_kext` names — before the grid is built, and the run prints the value it took. A positive value is taken as given, for reproducing a published normalization, and is then printed beside the grains' own number with the ratio, with a warning past 1%. A run with `ion_add_dust = .false.` carries no grains in the band and resolves a negative value to 0
`DGR`, `Z_global`, `Z_ref`	1e-2, 0.0134, 0.0134	dust-model scalings
`f_ion_dust`	0.01	laursen09 survival of dust in ionized gas
`f_pah`, `f_ion_pah`	0, 0	PAH share of the reference extinction and its own ionized-gas survival (laursen09_live)

2.3 Ionizing/FUV band and the source

parameter	default	meaning
`use_ion_band`	F	must be T (the MoCHII transport)
`nnu_ion`	16	diagnostic ionizing-energy bins (gates use 32). Packet energies are sampled from the source CDF and every cross section is evaluated at the sampled energy, so changing this count changes only the resolution of the saved tallies, never the physical solution
`eion_min`, `eion_max`	13.598, 100	ionizing band edges [eV]
`ion_align_edges`	T	align diagnostic bin edges to the ionization thresholds, so no bin straddles an edge; segments are log-uniform and low-priority metal edges are dropped safely when necessary. Affects the tally resolution only, not the transport. Set F for a single log grid
`add_fuv`	F	FUV extension: extra log bins on [`efuv_min`, `eion_min`] below the unchanged ionizing bins; transports FUV grain heating and FUV photoionization of low-threshold metals (Mg I, C I, S I, Fe I); `luminosity` stays the IONIZING luminosity (Q_H preserved, the FUV part rides on top)
`efuv_min`	6.0	FUV lower edge [eV]
`nnu_fuv`	8	FUV bins (16 recommended with metals)
`tstar`	–999	Planck source temperature [K]

parameter	default	meaning
ion_spectrum	”	global spectrum file (column 1 plus one value column); its column units are set by spectrum_type (§2.5)
spectrum_type	’shape_ev’	column units of every spectrum-file slot (ion_spectrum, src_spectrum_file, ext_spectrum): ’shape_ev’ (arbitrary, normalized to the scale — legacy) or a physical type ’per_ev’/’per_hz’/’per_ang’/’per_um’ (absolute); see §2.5
luminosity	unset	ionizing BAND luminosity [erg/s] of the (single) source; for a ’shape_ev’/Planck source unset maps to 1.0 (legacy), for a physical-type file unset means DERIVE it from the file integral; with nsource$\geq$ 2 it becomes the sum of src_lum (or the equal-split total when those are unset)
xs_point, ys_point, zs_point	0	single-source position (code units, recentered box; nsource=1)

Multiple internal point sources (the scalars above are the nsource=1 case):

parameter	default	meaning
nsource	1	number of internal point sources (0–16); 1 keeps the legacy scalars above, $\geq$ 2 uses the src_$\star$ arrays
src_x, src_y, src_z	0	positions of sources 1..nsource (code units)
src_lum	unset	ionizing luminosities [erg/s] of the sources; set all or none (none = equal split of luminosity); a partial set aborts
src_tstar	–999	Planck temperature [K] for each source (multi-source runs only; a single source uses tstar)
src_spectrum_file	”	one multi-column spectrum file for all sources: column 1 then one value column for each source (units set by spectrum_type, §2.5); each source is independently normalized so its ionizing segment carries src_lum(i) (a physical type with src_lum unset is derived from that column); overrides src_tstar; too few columns abort, extra columns are ignored with a note

External isotropic field (composes with the internal sources; ON when ext_intensity$>$ 0, an ISRF preset is named, or a physical-type ext_spectrum file is given):

parameter	default	meaning
`ext_intensity`	−1	band-integrated mean intensity J [erg s^{-1} cm^{-2} sr^{-1}] of the external field; the field is ON when > 0 and adds to the internal sources; entering ionizing power $L = \pi J A_{\rm surf}$. With a physical-type `ext_spectrum` file or an ISRF preset it is optional: unset = the absolute J is DERIVED from the file/preset, set = the field is RESCALED to this band J (a note is printed)
`ext_geometry`	'rec'	illuminating surface: `'rec'` (box faces, face-area-weighted — correct for a non-cubic box) or `'sph'` (bounding sphere of radius `rmax` about the box center; needs `rmax`> 0)
`ext_tstar`	–999	external-field Planck temperature [K]
`ext_spectrum`	"	external-field spectrum: a 2-column file (units set by `spectrum_type`, §2.5; column 2 is an intensity density J) OR a reserved ISRF preset name `'draine'`/`'habing'`/`'mathis'` (FUV-only, absolute, needs `add_fuv`); priority `ext_spectrum` > `ext_tstar` > the global spectrum
`ext_scale`	1.0	dimensionless multiplier on an ISRF preset (ignored when `ext_intensity`> 0, which rescales the preset to a target band J)
`source_geometry`	'point'	legacy external-only shorthand: `'external'`, `'external_rec'`, or `'external_sph'` forces `nsource=0` with the external field on (needs `ext_intensity`> 0); `'point'` keeps the composable form; `'slab'` is the plane-parallel slab illumination below

2.4 Plane-parallel slab illumination

With `source_geometry = 'slab'` (which requires `xy_periodic`, §2.2) the box is an *xy-periodic slab*: the medium varies in z (uniform in x, y for a true 1D column with `nx = ny = 1`, or an arbitrary 3D tile with `nx, ny` > 1 repeated periodically), and light enters through the top ($+z$) and/or bottom ($-z$) face. Each face is either a collimated *beam* at incidence angle θ or an *isotropic* (Lambert) field, with its own strength. The single physical normalization is the flux F_z crossing the (horizontal) illuminated face: for a beam $F_z = \mu F_n$ ($\mu = \cos\theta$, F_n the flux normal to the beam), for the isotropic mode $F_z = \pi I$ (I the specific intensity). F_z sets the packet weight through $L = F_z A_{\rm zface}$; every per-volume result is independent of the tile area. Point-observer peel-off is disabled under `xy_periodic` (the periodic images have no finite-distance observer); the emergent observable is the boundary intensity $I(\mu)$ (§3.3).

parameter	default	meaning
`slab_faces`	'top'	which z-face(s) are lit: `'top'`, `'bottom'`, or `'both'`

parameter	default	meaning
`slab_top_mode`, `slab_bot_mode`	'beam'	per face: `'beam'` (collimated) or `'isotropic'` (Lambert)
`slab_top_theta`, `slab_bot_theta`	0	beam incidence angle from the inward normal [deg]; 0 is normal incidence
`slab_top_phi`, `slab_bot_phi`	0	beam azimuth [deg]
`slab_top_source`, `slab_bot_source`	0	per-face source strength (band-integrated): the specific intensity I [erg s^{-1} cm^{-2} sr^{-1}] for `'isotropic'`, or the beam flux F_n [erg s^{-1} cm^{-2}] for `'beam'`
`slab_nmu`	20	number of μ-bins of the emergent $I(\mu)$ output

The spectrum is the global one (`tstar` / `ion_spectrum`). Both z-faces always escape (no reflecting boundary). `launch_sequence = 'sobol'` is supported (both modes). With dust scattering (`ion_dust_scatter`) the scattered/reflected component is included in the boundary $I(\mu)$.

2.5 Spectrum-file formats

A single parameter, `spectrum_type` (default `'shape_ev'`), fixes the column units of *every* spectrum-file slot (`ion_spectrum`, `src_spectrum_file`, `ext_spectrum`). The type sets the units; the slot sets the meaning: an internal-source file gives a luminosity density L_X, the external-field file gives an intensity density J_X. Column 1 and the value columns are:

`spectrum_type`	column 1	internal (cols 2+)	external (col 2)
`'shape_ev'`	E [eV] ↑	arbitrary per eV (shape of L_E)	arbitrary per eV
`'per_ev'`	E [eV]	L_E [erg s^{-1} eV^{-1}]	J_E
`'per_hz'`	ν [Hz]	L_ν [erg s^{-1} Hz^{-1}]	J_ν
`'per_ang'`	λ [Å]	L_λ [erg s^{-1} Å^{-1}]	J_λ
`'per_um'`	λ [μm]	L_λ [erg s^{-1} μm^{-1}]	J_λ

The external intensity density J_X carries the same spectral unit as the internal L_X times an extra cm^{-2} sr^{-1} (a mean intensity). All files are plain text with `'#'` comment lines; values outside the tabulated range are zero. `src_spectrum_file` is one multi-column file: column 1 shared by all sources, then one value column for each source (extra columns beyond `nsource` are ignored with a note, too few abort).

The abscissa is a sample grid, not bin centers. Column 1 lists the points at which the spectral density is *sampled*: the file describes a continuous function, defined between the points by linear interpolation, and no bin widths are attached to the rows. MoCHII integrates each of its own band bins on a 32-point sub-grid of the bin, interpolating the table at every sub-point (zero outside the tabulated range), so the file grid and the band grid are independent. The spacing is arbitrary — nonuniform grids are fine; place points densely where the spectrum varies rapidly, since only linear variation is captured between adjacent points. Abscissa values must be distinct (no duplicates); a `'shape_ev'` file must be ascending in E, while the physical types accept any row order (the table is sorted internally after the unit conversion, so a wavelength table may be given wavelength-ascending).

Absolute vs. shape normalization. A physical type (`'per_ev'`/`'per_hz'`/`'per_ang'`/`'per_um'`) is *absolute*: the bin luminosities follow the tabulated integral directly. If the matching scale (`luminosity` / `src_lum`(i) / `ext_intensity`) is unset it is DERIVED from the file — each source from its column's ionizing-segment integral, the external field from its band-integrated J — and reported in the log. If a scale is set, the spectrum is RESCALED to it (a note is printed) and the file then acts only as a shape. With `add_fuv` an absolute spectrum fills the FUV bins from the same table, with no separate normalization. `'shape_ev'` keeps the legacy behavior: the shape is normalized to the scale in every case. A spectrum with no luminosity inside the band aborts, as does rescaling a file whose ionizing-segment integral is zero.

`'shape_ev'` is per eV, not variable-agnostic. `'shape_ev'` reads its value columns as the shape of the per-eV density L_E on the E [eV] grid of column 1. A table of F_λ (or F_ν) values re-gridded onto E under `'shape_ev'` therefore omits the Jacobian and is wrong by a factor E^{-2} (the $\mathrm{d}\lambda/\mathrm{d}E$ conversion). To use a wavelength- or frequency-tabulated shape, give it with its matching physical type and set the scale explicitly — the type conversion then applies the Jacobian and the scale RESCALES the file to your target, so the file acts as a shape with the correct units. For example, a wavelength shape normalized to 10^{49} ionizing photons' worth of luminosity:

```
par%spectrum_type = 'per_ang'
par%ion_spectrum  = 'wave_shape.txt'
par%luminosity    = 3.0e38
```

A small `'per_ang'` source file for two point sources (λ ascending; the second source is FUV-faint):

```
# lambda[A]   L_lam,1        L_lam,2
   300.0      7.80e34        3.4e33
   500.0      6.05e34        1.2e33
   912.0      4.10e34        0.0
```

2.6 Initial state and iteration

parameter	default	meaning
`xHI_init`	1.0	initial $n_{\mathrm{HI}}/n_{\mathrm{H}}$ (an `xHI` file column wins); an ionized start (1e-3) converges much faster than neutral
`xHeI_init`, `xHeII_init`	1, 0	initial He fractions
`abund_He`	0.1	$n_{\mathrm{He}}/n_{\mathrm{H}}$
`gas_niter`	0	iterations (0 = a single rates-only pass)
`gas_tol`	1e-3	convergence tolerance on x_{HII} (measure set by `conv_crit`)
`conv_crit`	'cell'	convergence measure: 'cell' (cell maxima; stalls at the front-cell noise floor) or 'vol' (volume-integrated L_1; noise-robust, recommended). See §2.7
`ion_relax`	1.0	under-relaxation on x (< 1 damps front oscillation)
`case_ab`	'B'	case A/B recombination in the ionization balance, i.e. how the ionizing *continuum* is treated ('A' is forced by `diffuse_field`, which transports the ground-state recombination photons)

parameter	default	meaning
`line_case`	'B'	case of the SH95 H I/He II recombination-*line* tables. This is set by the optical depth of the Lyman *lines*, a different question from `case_ab`: a nebula whose diffuse continuum is transported still traps its Lyman lines and radiates case-B Balmer lines, so 'B' is the physical default for H II regions. 'A' selects the case-A tables (Lyman-line-thin gas); 'follow' ties the line case to `case_ab`
`require_convergence`	F	if the gas iteration has not converged at `gas_niter`, print a warning; when T, abort before writing output. The convergence status is recorded in the `_rates` header either way (`CONVERGD`, `NITERDON`, `FINALDX`, `FINALDTE`)

2.7 Convergence of the gas iteration

The gas iteration (`gas_niter` sweeps of transport → rate integrals → leaf solve → opacity refill) stops when the state stops changing between sweeps. Each sweep compares a *change measure* against a tolerance:

- ionization only (`te_fixed`, no thermal solve): converged when the $x_{\rm HII}$ change $<$ `gas_tol`;
- coupled (`solve_te`): converged when the $x_{\rm HII}$ change $<$ `gas_tol` *and* the T_e change $<$ `gas_tol_te`.

Two measures (`conv_crit`). Both are computed every sweep; `conv_crit` selects which one gates the loop.

`'cell'` the cell maxima $\max_{\rm leaf}|\Delta x_{\rm HII}|$ and $\max_{\rm leaf}|\Delta T_e|/T_e$. Simple, but the ionization-front cells jitter with Monte Carlo noise and never settle, so $\max|\Delta x_{\rm HII}|$ *stalls* at the front-cell noise floor ($\sim$2–4$\times10^{-2}$ at 4×10^7 packets) and a smaller `gas_tol` is unreachable. This is the historical default (kept so the recorded gates reproduce).

`'vol'` the volume-integrated L_1 changes

$$\frac{\sum_{\rm leaf} V\,|\Delta x_{\rm HII}|}{\sum_{\rm leaf} V\,x_{\rm HII}} < \texttt{gas_tol}, \qquad \frac{\sum_{\rm leaf} V\,n_e\,|\Delta T_e|}{\sum_{\rm leaf} V\,n_e\,T_e} < \texttt{gas_tol_te}.$$

Front jitter occupies little volume (a few thousand front cells against $\sim10^5$ ionized ones), so it enters the norm suppressed by that ratio; the n_e weight drops the vacuum and `te_min`-pinned no-heating cells that otherwise dominate $\max|\Delta T_e|$ in FUV/PDR runs. These measures decay smoothly — the recommended production setting.

The Monte Carlo floor. Even the volume measures do not reach zero: they floor at a Monte Carlo level set by the packet count, scaling as $1/\sqrt{N_{\rm packets}}$ ($\sim3\times10^{-3}$ in x and $\sim7\times10^{-3}$ in T_e at 8×10^6 packets). Reaching the floor *is* convergence — iterating further leaves the solution unchanged. So **set `gas_tol`/`gas_tol_te` just above the floor for the packet count in use**: a tolerance below the floor can never be met and the loop runs to `gas_niter` and warns. Rules of thumb (`conv_crit='vol'`): $\sim5\times10^{-3}/10^{-2}$ at $\sim10^7$ packets, loosened to $\sim10^{-2}/2\times10^{-2}$ at $\sim10^6$ (the reduced-count `examples/` use the looser values for this reason).

When it does not converge. Hitting `gas_niter` without meeting the test is not fatal: the state is still written and a warning is printed on rank 0. Set `require_convergence = T` to abort before the output instead. Either way the `_rates` header records the outcome in `CONVERGD` (0/1), `NITERDON` (sweeps run), `FINALDX`, and `FINALDTE`. The derivation and the gate measurements are in `docs/MoCHII_physics.pdf`.

2.8 Thermal balance and metals

parameter	default	meaning
`solve_te`	T	bisection on $\log T_e$ in [`te_min`, `te_max`] coupled with the ionization (heating = cooling in every leaf); F holds every leaf at `te_fixed`
`te_fixed`	1e4	T_e [K] of every leaf when `solve_te` is off; with it on, only the starting T_e of the first pass
`te_min`, `te_max`	1e3, 5e4	bisection bracket [K]
`gas_tol_te`	1e-3	convergence on $\max \lvert\Delta T_e\rvert / T_e$
`cooling_model`	'local_ne'	line cooling in the thermal loop: 'local_ne' (default) suppresses each ion's cooling fit to the local n_e with a (T, n_e) table built from its n-level solve, so the thermal balance and the emergent lines are one physics; 'low_density' uses the $n_e \to 0$ fits directly (the coronal limit, exact in diffuse gas; reproduces earlier gates)
`atomic_dir`	'data/atomic'	coefficient files (relative to the run directory)
`use_metals`	F	registry cascade + metal line cooling
`ci_model`	'voronov'	collisional ionization: 'voronov' (default) or 'dere_hybrid' (Voronov $\times$ the Dere 2007/Voronov ratio; matters only in shielded low-ionization zones)
`abund_C/N/O/Ne/S`	HII20 values	$n(\mathrm{X})/n(\mathrm{H})$; an element is active when > 0
`abund_Ar/Mg/Fe`	0	further registry elements (gas-phase values; Mg/Fe are heavily depleted)
`abund_Si/Cl/Ca`	0	five-stage registry elements (gas-phase; Si and especially Ca heavily depleted, Cl nearly undepleted — Orion-like 3e-6 / 3e-7 / 2e-8); adds [Si II] 34.8 μm, Si III/IV, [Cl II/III/IV] (5518/5538 density pair), Ca II, [Ca V]
`ion_metal_abs`	T	metal photoionization absorption in the band opacity (active with `use_metals`); the only gas opacity in the FUV bins

parameter	default	meaning
`charge_exchange`	'two_way'	charge exchange $X^{i+} + H^+ \leftrightarrow X^{(i+1)+} + H^0$ in hydrogen's balance as well as the cascade's, so the charge each reaction moves is conserved in the reacting pair: $n_{HI}(\Gamma_H + n_e C_H + R_{cx}^{out}) = (\alpha_H n_e + R_{cx}^{in}) n_{HII}$ with the sums over all 29 active reactions. 'metal_only' keeps them in the cascade alone (the earlier bookkeeping, which does not conserve charge); it is a reproduction value, not a physics option

parameter	default	meaning
`emis_output`	F	write `_emis` (HDF5/FITS): leaf-by-leaf line emissivities + the state needed for maps (Section 3.)
`hei_metastable`	F	write `_heimeta` (HDF5/FITS): the He I 2^3S metastable density n_3, its photoionization rate Φ_3, plus n_H, n_e, T_e, and leaf geometry (the 10833 Å absorption diagnostic; Section 3.). With `add_fuv` and `efuv_min`=4.78 the soft-UV part of Φ_3 is included
`h_coll_effects`	F	add the collisional H I Balmer component (excitation of neutral H) as separate `hc` rows/blocks; matters at fronts and hot cells
`fe_levels_full`	F	load the expanded Fe II/III models (`nlevel_fe_{2,3}_full.txt`, 16/34 levels) instead of the compact 13/14 for iron-line studies
`diffuse_field`	F	explicit ground-recombination packets
`diffuse_energy_model`	'milne'	spectrum of the three ground continua (needs `diffuse_field`): 'milne' (default) is the detailed-balance free–bound spectrum $E^2\sigma_{pi}(E)e^{-(E-E_{th})/kT}$ with the transport cross sections; 'exponential' the historical $E_{th} + kT[-\ln u]$ approximation (reproduces earlier runs); 'threshold' a sensitivity bracket, not a model

parameter	default	meaning
`hei_diffuse`	F	fourth diffuse channel (needs `diffuse_field`): He I excited-level recombination photons. The $2^3S : 2^1S : 2^1P$ branch fractions come from one consistent set of $\alpha_{\rm eff}$ (Benjamin, Skillman & Smits 1999 Table 1 with J. S. Mathis, as in K. Wood's `ionize`) normalized by its own sum — 0.762 : 0.076 : 0.162 at 8000 K against the statistical 0.750 : 0.083 : 0.167 — and the 2^1S photons from the Drake et al. (1969) two-photon shape. Correct energy bookkeeping: +0.12% in energy and +0.30% in photons per case-B He II recombination, $L(\mathrm{H}\beta)$ within 0.02%, He split unchanged. The trapped 2^1P (584 Å) photon is split locally between photoionizing hydrogen and collisional conversion $2^1P \rightarrow 2^1S$, with $p_{584} = [1 + 77x(\mathrm{He}^0)/(\sqrt{T}x(\mathrm{H}^0))]^{-1}$ (Wood, Mathis & Ercolano 2004) — the same split the He I two-photon nebular continuum uses. At HII40 it lowers $L(\mathrm{H}\beta)$ by 0.2% and moves the He I front inward by 0.0036 pc
`heii_diffuse`	F	fifth diffuse channel (needs `diffuse_field`): what a He III case-B recombination radiates on its way down through He II. Unlike `hei_diffuse` this is not a small correction — the He II $n = 2$ threshold at 13.604 eV sits just above the hydrogen edge, so the whole Balmer continuum ionizes H, and the 2^2S pair spans 40.813 eV. Three paths: free–bound into $n \geq 2$ (7.4–10.2% of the channel over 10^4–3×10^4 K, from the same fb+ff tables the written continuum uses, minus free–free), 2^2P Lyα at 40.813 eV (64.2–70.0%), and the 2^2S two-photon continuum (22.5–25.6%, 36.090 eV of the pair landing in band). The $n = 2$ exit branching $f_{2s} = \alpha_{\rm eff}(2^2S)/\alpha_B$ takes both parts from Storey & Hummer (1995) so the ratio closes one cascade; it reproduces MOCASSIN's independent He II Lyα fit to 2.3–6.1%. Matters only where He^{2+} exists: on a 10^5 K blackbody in the HII40 geometry it raises the diffuse-field luminosity 44% and $L(\mathrm{H}\beta)$ 0.98%; on both Lexington benchmarks it changes nothing
`gaunt_vh14`	F	free–free Gaunt factors from the van Hoof et al. (2014) table (`data/gauntff_vh14.dat`) instead of Hummer (1988); the two agree to $\lesssim$0.5% in T_e

parameter	default	meaning
`metal_ne`	T	electrons from the metal cascade join the n_e closure. Solved as a fixed point, since the stage fractions depend on n_e themselves: a single evaluation at the H/He-only electron density would be the $n_e \to 0$ limit, in which nothing recombines and every low-IP metal comes out fully ionized. Switching it off costs only 0.002% of $\langle T[N_\mathrm{p}N_\mathrm{e}]\rangle$ on the Lexington benchmarks — but beyond the ionization front the metal stages are the ONLY electron source, so there it is not optional
`metal_heat`	T	metal photoheating in the thermal balance, from the same Γ's the cascade uses. Switching it off lowers $\langle T[N_\mathrm{p}N_\mathrm{e}]\rangle$ by 21.1 K (0.26%) at HII40 and 5.7 K (0.082%) at HII20
`grain_pe`	F	PDR thermal package: grain photoelectric heating + recombination cooling (Bakes & Tielens 1994) from the local FUV field (needs `add_fuv`), plus H-impact [C II] 158 μm / [O I] 63 μm cooling; scaled by `pe_scale` and the leaf dust amount
`pe_scale`	1.0	photoelectric-efficiency scale factor
`use_sec_ion`	F	secondary ionization by fast photoelectrons (Shull & van Steenberg 1985): a photoelectron with kinetic energy $E_0 = h\nu - E_\mathrm{th}$ above 40 eV deposits only $f_\mathrm{heat}(x)$ of its excess as heat and the balance drives secondary H I/He I ionizations (x = ionized fraction of the H+He nuclei); $\sim$0 in fully ionized gas, matters at fronts / PDR / hard fields
`metal_freefree`	T	include the trace metals in the free–free cooling with the net ion charge Z_ion = (stage −1), not the nuclear Z; a trace ($\sim 10^{-4}$ of the H/He free–free) correctness term, on by default

2.9 I-front re-refinement

parameter	default	meaning
`refine_front`	F	rebuild the octree on the I-front once
`refine_iter`	20	at which iteration
`refine_lbase`, `refine_lmax`	5, 7	base/front levels
`refine_eps`	1e-2	front flag: $\epsilon < x_\mathrm{HI} < 1-\epsilon$
`refine_dx`	0.2	or a face-neighbor x_HI jump above this

2.10 Dust in the band and dust emission

parameter	default	meaning
ion_add_dust	F	dust absorption in the band (dusty Strömgren)
ion_dust_kext	”	optional kext table (e.g. data/kext_albedo_WD_MW_3.1_60_D03.all_2003). Blank (the default) takes $C_{\rm ext}$, albedo and $\langle\cos\rangle$ from the grain model named by dust_model, so the transport and the emission refer to the same grains; a file OVERRIDES the model and is then consistent with the emission only if it came from the same grains. Taken from the model, the curve is /kext of that model's own optics product, SEDust/data/<model>/sedust_<model>.h5, which carries the wavelength axis and the cross-section tables in the same file, interpolated onto the model's own grid (sed_kext replaces it). The model must cover the band, and all four do on their own: astrodust spans 1762 wavelengths over 1.0×10^{-4}–3.981×10^{4} μm from the T-matrix optics table, so below the Lyman limit the optics are the $b/a = 1.4$ oblate spheroid itself rather than a sphere approximation; DL07 and MRN share their own axis, 1823 wavelengths from 6.2×10^{-5} μm; and Zubko's DustEM optics span 0.001–10^{4} μm. Nothing is extended at any energy the code transports; an extension can only be asked for above eion_max = 12398.4 eV, and astrodust refuses it there with a message naming eion_max rather than substitute an equal-volume sphere
ion_dust_scatter	F	sample dust scattering (HG, albedo weights)
dust_sed	F	SEDust stochastic dust SED at output time
dust_model	'astrodust'	'astrodust'/'dl07'/'mrn'/'zubko'. 'mrn' is the Mathis, Rumpl & Nordsieck (1977) graphite + silicate $a^{-3.5}$ power law over 0.005–0.25 μm on the Draine & Lee (1984) abundances, Mie on the D03 dielectric functions; it has NO PAH component, so a run on it carries no aromatic features and sed_pah_live has nothing to weight

parameter	default	meaning
`sed_data_dir`	” (auto)	the ONE directory everything the named `dust_model` is made of is resolved from: its optics product `<model>/sedust_<model>.h5`, the size distribution under `release/`, the dielectric functions and PAH cross sections under `dielectric/`, and Zubko's ZDA configuration. Blank auto-resolves to `<executable dir>/SEDust/data`, so a fresh checkout runs the dust path from any working directory
`sed_dl07_sdindex`, `sed_dl07_uisrf`	7, 1.0	DL07 only: WD01 size-distribution index (7 = MW $R_V = 3.1$, $b_C = 6\times10^{-5}$) and the ISRF intensity setting the PAH ionization fraction. Leave them at the defaults unless you also supply a matching `sed_kext`: the served extinction curve was computed from the defaults, so moving these moves the grains that emit without moving the curve that extinguishes. The gate `tests/grain_kext` checks that pairing
`sed_kext`	”	the precomputed size-integrated extinction curve SEDust serves to the transport (λ, albedo, $\langle\cos\rangle$, $C_{\rm ext}$/H, $C_{\rm abs}$/H, $C_{\rm sca}$/H), resolved under `sed_data_dir`. Blank (the default) takes `/kext` of the named `dust_model`'s own product, so the curve and the cross sections come from one file and cannot be paired wrongly. Naming a file makes it MANDATORY (the build stops if it cannot be read) and it must have been computed from the same model and size distribution the emission uses, which is what SEDust's `calc_kext.x` writes. Distinct from `ion_dust_kext`, which does not change what SEDust serves but REPLACES the served curve downstream, leaving the emission on other grains
`sed_NT`, `sed_Tlo`, `sed_Thi`	200, 2.7, 5e3	SEDust temperature grid
`dust_emis_bands`	unset	up to 8 wavelengths [μm] (e.g. 8, 24, 70, 160): SEDust band emissivities of each leaf → `_dustemis` (blocks `emis_dust/wl_dust` [erg s^{-1} cm^{-3} μm^{-1}]); the Python map maker projects them into dust-band images (the IR is optically thin)

parameter	default	meaning
sed_pah_live	F	$x_{\rm HII}$-dependent PAH survival in the emission: the leaf spectrum is assembled from the model channels with the PAH channel weighted by $(x_{\rm HI} + f_{\rm ion,pah}\, x_{\rm HII})$ (needs a model with a 'PAH' channel, i.e. astrodust); turns the 8-μm map into the classic PAH ring at the ionization front
dust_emis_transport	T	transport the reradiated infrared through the same grain opacity, launching from each leaf in proportion to its emitted power with the wavelength drawn continuously from that leaf's spectrum; what the packets deposit returns to Heat_dust. Set F to post-process the emission only
dust_niter, dust_tol	2, 1e-3	maximum transport passes and the relative change in emitted luminosity that ends them; = 1 transports once with no self-consistency check
dust_no_photons	1e6	infrared packets per pass

2.11 Peel-off imaging

parameter	default	meaning
ion_peel	F	after convergence, one extra transport pass peels direct (stellar + diffuse) and dust-scattered contributions toward the observers → _image (blocks direct_ion/scatt_ion, plus _fuv with add_fuv) [erg s^{-1} cm^{-2} sr^{-1}]
peel_bins	F	bin-resolved cubes direct_cube/scatt_cube (n_x, n_y, n_ν) in addition to the band-integrated channel images (hardness maps, synthetic EUV/FUV photometry); the third (bin) axis is labeled by the E_bin/dE_bin [eV] and wl_bin [Å] arrays written once into the same file
save_direc0	F	also write the unattenuated direct image (direc0_ion/_fuv, and direc0_cube with peel_bins): the direct source with the path extinction $e^{-\tau}$ omitted, so direc0/direct= $e^{+\tau}$ is the line-of-sight optical depth toward the observer
obsx,obsy,obsz	unset	observer positions (or directions) in code units; or use alpha/beta/gamma angle lists [deg]; default: one observer on +z far away
distance	auto	observer distance (code units)

parameter	default	meaning
nxim, nyim	129, 129	image pixels
dxim, dyim	auto	pixel scale [deg]; auto-sized to cover the box

Peel-off also works with the external field: an external packet's direct peel uses the Lambert weight $\max(\hat{k}_{\rm obs}\cdot\hat{n}_{\rm in},0)/\pi$ about its entry-surface inward normal, so only the far-side surface contributes and the image is the background field seen through the medium (the cloud in silhouette); with save_direc0, direc0/direct = $e^{+\tau}$ is again the line-of-sight optical-depth map.

2.12 Output

out_file (base name; extension follows file_format = 'hdf5' or 'fits').

3. Output files

3.1 <base>_rates.h5

One dataset for each extension, leaf-ordered:

extension	content
Gamma_HI/HeI/HeII	photoionization rates [s^{-1}]
Heat_HI/HeI/HeII	photoheating per particle [erg/s]
Heat_dust	grain heating [erg s^{-1} cm^{-3}] (when ion_add_dust)
T_dust	equilibrium mixture dust temperature [K]
x_HI, x_HeI, x_HeII, n_e, T_e	converged state
x_<el>_stages	metal ion fractions (stage, leaf)
J_nu	band mean intensity (bin, leaf) [erg/s/cm^2/Hz/sr]
E_bin, dE_bin	band grid [eV]
LeafXYZ, LeafSize	leaf centers and full widths (code units; for slices/cutouts)

The file header also carries the convergence record of the gas iteration: CONVERGD (did it converge), NITERDON (iterations run), FINALDX (final max $|\Delta x_{\rm HII}|$), and FINALDTE (final max $|\Delta T_e|/T_e$).

3.2 Line and emissivity outputs

<base>_lines.txt is a text table of integrated line luminosities on the converged state, one row for each line: element, ionization stage, wavelength [Å], L [erg s^{-1}], and $L/L(\mathrm{H}\beta)$. It is written by every run: the H and He recombination lines need only n_e, T_e and the He ionization state, so an H/He model at fixed temperature gets its table too, with the metal blocks simply absent. With emis_output the same line set is also written leaf-by-leaf to <base>_emis (HDF5, or FITS with file_format='fits') as emissivity blocks in erg s^{-1} cm^{-3}, where $\sum \mathrm{emis}\cdot V = L_{\rm line}$ and the block row order

matches the text table. Every line comes from one of two sources: recombination-line tables (H and He, Table 14) and n-level statistical-equilibrium solves (the metals plus the optional collisional H I component, Table 15).

Table 14: Recombination-line blocks in `<base>_emis`; wavelengths in Å unless μm noted.

block	source	lines	weighting, when present
`emis_h`	Storey & Hummer (1995) H I, case set by `line_case`	Hα 6565, Hβ 4863, Hγ 4342, Hδ 4103, Paα 1.876 μm, Paβ 1.282 μm, Brγ 2.166 μm	always; $n_e\, n_{\mathrm{HII}}$
`emis_heii`	SH95 He II (case per `line_case`)	1640, 3204, 4543, 4687, 5413, 10126	only where an He III zone exists; $n_e\, n_{\mathrm{HeIII}}$
`emis_hei`	Porter et al. (2012/2013) He I, case-B collisional-radiative (density-dependent)	3890, 4027, 4473, 5877, 6680, 7067, 7283, 10833	where He II is present; $n_e\, n_{\mathrm{HeII}}$
`emis_hc`	25-level H I collisional solve	Hα/Hβ/Hγ collisional component (separate `hc` rows)	only with `h_coll_effects`; n_{HI}

The He I Porter table is a case-B collisional-radiative total (density-dependent; it already includes collisional excitation from the 2^3S metastable), case B only; under case A a note is printed and the H I/He II tables switch to case A while He I stays case B.

The metal line blocks `emis_<el>_<stage>` follow the n-level solve. One block is written for every active registry ion (element abundance > 0) that has an `nlevel_<el>_<stage>` data file. Elements C, N, O, Ne, S are active by default; Ar, Mg, Fe, Si, Cl, Ca are activated by setting their `par%abund_<El>`. Only the principal lines are listed below — the full transition list of each block is in `<base>_lines.txt`.

Table 15: Metal collisional-line blocks (`emis_<el>_<stage>`); wavelengths in Å unless μm noted.

ion (block)	principal lines
`c_2` [C II]	2325 (UV multiplet), 158 μm
`c_3` C III]	977, 1907, 1909, 178 μm
`n_2` [N II]	3064/3071, 5756, 6550, 6585, 122 μm, 205 μm
`n_3` [N III]	1750 (UV multiplet), 57 μm
`o_1` [O I]	5579, 6302, 6366, 63 μm, 145 μm
`o_2` [O II]	2471, 3727, 3730, 7321/7331
`o_3` [O III]	1661/1666, 2322/2332, 4364, 4960, 5008, 52 μm, 88 μm
`ne_2` [Ne II]	12.8 μm
`ne_3` [Ne III]	3343, 3870, 3969, 15.55 μm, 36 μm
`s_2` [S II]	4070/4078, 6718, 6733, 1.03 μm, 315 μm
`s_3` [S III]	3722, 6314, 8831, 9070, 9532, 12 μm, 18.7 μm, 33.5 μm

Table 15 (continued)

ion (block)	principal lines
Optional — activate with `par%abund_<El>`	
`ar_3` [Ar III]	3006/3110, 5193, 7138, 7753, 8.99 μm, 21.8 μm
`ar_4` [Ar IV]	2854/2869, 4713, 4741, 7170 (multiplet), 56 μm, 78 μm
`mg_2` Mg II	2796, 2804
`fe_2` [Fe II]	1.26/1.32 μm, 5.34 μm, 17.9 μm, 26 μm (40 lines total)
`fe_3` [Fe III]	4659, 4703, 4881, 5011, 22.9 μm (38 lines; 16/34-level with `fe_levels_full`)
`si_2` [Si II]	2335 (multiplet), 34.8 μm
`si_3` Si III]	1206, 1883, 1892, 38 μm
`si_4` Si IV	1394, 1403
`cl_2` [Cl II]	3587/3679, 6164, 8581, 9126, 14.4 μm, 33 μm
`cl_3` [Cl III]	3344/3354, 5519, 5539, 8480 (multiplet)
`cl_4` [Cl IV]	3120/3205, 5325, 7263, 7533, 11.8 μm, 20.3 μm
`ca_2` Ca II	3935, 3970, 7293/7326, 8500/8544/8665
`ca_5` [Ca V]	3999, 5311, 6088, 3.05 μm, 4.16 μm, 11.5 μm

Some registry ions are tracked in the ionization balance but carry no line block because CHIANTI v11 has no level/collision data for them: Ar II, Cl V, Ca III, Ca IV.

State blocks. Beyond the emissivities, `<base>_emis` carries the state a map needs without the rates file: `LeafXYZ` (leaf centers) with the `DIST_CM` keyword, `LeafSize` (leaf edge length; $V = (\texttt{LeafSize}\cdot\texttt{DIST_CM})^3$), `n_H`, `n_e`, `T_e`, `x_HI`, `x_HeI`, `x_HeII`, and the `x_<el>_stages` metal-fraction blocks (same layout as the rates file). The Python `EmisData` reader (`tools/python/mochii_output.py`) turns these blocks into line maps without the rates file.

3.3 Spectral and image outputs

file	content
`_nebcont.txt`	nebular continuum L_ν (fb+ff tables, Milne, two-photon)
`_dustir.txt`	equilibrium-mixture dust IR spectrum (1–1000 μm) with the energy-closure header
`_dustsed.txt`	SEDust stochastic dust SED with PAH bands (when `dust_sed`)
`_dustesc.txt`	(with `dust_emis_transport`) the infrared spectrum that actually escapes the grid, from the transported packets. Its ratio to the emitted spectrum in `_dustir.txt`/`_dustsed.txt` is the infrared escape fraction
`_dustemis.h5`	(with `dust_emis_bands`) SEDust band emissivities of each leaf, same block layout as `_emis` — readable by `EmisData` for dust-band maps
`_heimeta.h5`	(with `hei_metastable`) the He I 2^3S diagnostic: `n_2s3` [cm^{-3}] (with the `DIST_CM` and `A31` keywords), `Phi_3` [s^{-1}], `n_H`, `n_e`, `T_e`, `LeafXYZ`, `LeafSize` — read by the Python `HeiMetaData` class for column and 10833 Å optical-depth maps, and convertible to a LaRT scattering-medium input by `heimeta_to_lart.py`
`_image.h5`	(with `ion_peel`) peel-off images for each observer: `direct_ion`/`scatt_ion` (+`_fuv`) [erg s^{-1} cm^{-2} sr^{-1}], suffix `_obs<k>` when several observers; with `peel_bins`, `direct_cube`/`scatt_cube` (n_x, n_y, n_ν) and the `E_bin`/`dE_bin`/`wl_bin` bin-grid arrays; with `save_direc0`, the unattenuated `direc0_ion`/`_fuv` (or `direc0_cube`)
`_slab_Imu.txt`	(with `source_geometry = 'slab'`) the emergent intensity $I(\mu)$ at the two z-boundaries: columns μ, $I_{\rm top}$, $I_{\rm bot}$ [erg s^{-1} cm^{-2} sr^{-1}] on `slab_nmu` bins ($F = 2\pi\int I\mu\,\mathrm{d}\mu$), with the reflected / transmitted / absorbed energy budget (fractions of the entering L) in the header

3.4 Reading outputs in Python

`tools/python/mochii_output.py` is an importable module (works directly in a notebook) that reads every section file (HDF5 or FITS; `read_sections`), the line table (`read_lines_table`), and the text spectra (`read_spectrum`), and builds 2D maps from the emissivity file:

```
import sys; sys.path.insert(0, ".../MoCHII/tools/python")
import matplotlib.pyplot as plt
from mochii_output import EmisData

d = EmisData("run_emis.h5")
d.info()                          # blocks, fields, line count
d.line_list()                     # every stored line (block, wl)
```

```
# ready-made figures (notebook): returns (fig, ax)
fig, ax = d.plot_line("o_3", 5007.)                  # log S map
fig, ax = d.plot_line("h", 6563., unit="intensity",
                      photons=True)  # photons/s/cm^2/sr
fig, ax = d.plot_field("T_e", weight="EM")
fig, axs = plt.subplots(1, 2); d.plot_line("s_2", 6717., ax=axs[0]) \
    ; d.plot_line("s_2", 6731., ax=axs[1])   # doublet side by side

# raw arrays, when the figure is yours to build
img, ext = d.line_map("o_3", 5007., axis="z", npix=512)
img, ext = d.line_map("h", 6563., unit="intensity", photons=True)
img, ext = d.field_map("T_e", weight="EM")   # LOS-weighted average
Q = d.line_luminosity("h", 4861., photons=True)  # photon rate [1/s]
```

Line maps are flux-conserving: each leaf's luminosity is deposited with exact area overlap onto the pixel grid (AMR leaf sizes handled), so $\sum SA_{\rm pix} = L_{\rm line}$ to machine precision. `unit='flux'` (default) gives surface brightness [erg s^{-1} cm^{-2}]; `unit='intensity'` gives specific intensity [erg s^{-1} cm^{-2} sr^{-1}] $= S/4\pi$ (isotropic emission, optically thin); `photons=True` converts either to photon units (divides by hc/λ of the line). A `slab=(lo,hi)` cut restricts the line of sight. Field maps average `T_e`, `n_e`, `x_HI`, … along the line of sight with weight `'EM'` ($n_e^2 V$), `'ne'`, `'nH'`, or `'V'`. The same module doubles as a command-line tool for quick looks:

```
python3 mochii_output.py run_emis.h5                  # list lines
python3 mochii_output.py run_emis.h5 --line o_3 5007 \
        --npix 512 --log --out o3.png
python3 mochii_output.py run_emis.h5 --line h 6563 \
        --unit intensity --photons     # photons/s/cm^2/sr
python3 mochii_output.py run_emis.h5 --field T_e --weight EM
```

3.5 Slices and cutouts

A *slice* is the actual leaf value on a plane (the cell that contains each pixel), distinct from the line-of-sight *projection* above — a slice resolves the ionization front sharply where a projection blurs it. `tools/python/leaf_field.py` holds the backend (`leaf_slice`, the nearest-leaf fallback `leaf_slice_nn`, and `plot_slice`); `mochii_output.py` exposes it on the output files:

```
from mochii_output import EmisData, RatesData
r = RatesData("run_rates.h5")                # leaf state
fig, ax = r.plot_field_slice("T_e", axis="z", coord=0.0)
fig, ax = r.plot_field_slice("x_HI", axis="z", coord=0.0, log=True)
img, ext = r.slice_field("n_e", axis="x", coord=0.0,
                         bounds=(-2,2,-2,2))  # cutout window
```

```
d = EmisData("run_emis.h5")
fig, ax = d.plot_line_slice("o_3", 5007., axis="z")   # emissivity slice
```

Exact slices need the leaf full widths: the `_emis.h5` file always carries `LeafSize`, and `_rates.h5` does too; a rates file written before `LeafSize` existed falls back to a nearest-leaf slice (a one-time note is printed). The command line takes `--slice FIELD --axis z --coord 0` on either file.

3.6 He I 2^3S metastable and 10833 Å absorption

The `_heimeta` file (`hei_metastable`) is read by the `HeiMetaData` class, which adds a line-of-sight column map and the 10833 Å line-center optical depth:

```
from mochii_output import HeiMetaData
d = HeiMetaData("run_heimeta.h5")
d.info()                                  # n_2s3, Phi_3, line constants
img, ext = d.column_map("n_2s3")          # LOS column [cm^-2]
img, ext = d.tau10833_map(v_turb=0.0,     # tau_0 map (km/s turbulence)
                          component="doublet")  # blended J'=1,2 pair
fig, ax = d.plot_column("n_2s3", log=True)
fig, ax = d.plot_tau10833(component="all")
```

`column_map` is the same flux-conserving deposit as the line maps with weight nV, so $\sum NA_{\rm pix} = \sum nV$ to machine precision. `tau10833_map` deposits the line-center opacity $\kappa_0 = \frac{\sqrt{\pi}e^2}{m_e c} f \frac{\lambda}{b} n_3$, $b = \sqrt{2kT_e/m_{\rm He} + v_{\rm turb}^2}$, so a pixel value is $\int \kappa_0 \, {\rm d}l$. `component` selects the resolved $J' = 0$ line (10832.06 Å), the blended $J' = 1,2$ `'doublet'` (10833.22/10833.31 Å, 0.089 Å apart, inside the thermal Doppler width), or `'all'`; the constants (λ, f, A) come from `data/atomic/hei_metastable.txt`. The map warns when line-center $\tau_0 > 1$: the diagnostic assumes optically thin 10833 (line transfer is out of scope). The command line takes `--column n_2s3` or `--tau10833 --component doublet --vturb 5`.

Where line transfer *is* wanted (resonant-scattering emission profiles from the metastable medium), `heimeta_to_lart.py` (in `tools/python/`) converts a `_heimeta` file into a scattering-medium input for the resonance-line code LaRT (line `HeI_10833`): `--format car` (default, uniform grids) writes the all-in-one Cartesian file with `gasDen` = n_3, `T` = T_e, and zero velocities; `--format amr` writes the generic AMR table with n_3 in the `n_ion` column. `--field` selects a different leaf field for the scatterer. A worked end-to-end case (converted grid + LaRT namelist) is in `examples/hei_metastable/lart/`.

3.7 Building AMR grids

`tools/python/AMR_grid/` builds the generic AMR octree file the code reads (`grid_type='amr'`): `AMR_grid.py` (the `AMRGrid` class — uniform, density-gradient, or resolution refinement; `set_density`/`set_neutral_fraction`; `write` to HDF5/FITS/`.fits.gz`/text with columns `x,y,z,level,nH[,metallicity,xHI,ndust]`) and `make_amr_sphere.py` (a sphere/shell CLI). `AMRGrid` carries the same `slice`/`plot_slice`/`cutout` so a grid can be inspected before a run.

4. Worked examples (the regression tests)

Each `tests/` directory is self-contained: grid builder or a symlinked grid, the input file, and a `check_*.py` gate script.

directory	what it demonstrates / gate
`g0_gamma`	rate integrals vs. analytic attenuation (bias +0.004%)
`g1_stromgren`	Strömgren sphere, case B fixed T_e; AMR ≡ uniform
`g2a_thermal`	thermal H/He nebula vs. the 1D exact reference
`g2_hii`	MOCASSIN Lexington HII20/HII40 benchmarks + line fluxes
`g4_refine`	I-front re-refinement vs. native level 7
`g4_tng`	Illustris-TNG post-processing demo (shadows, maps)
`d_dusty`	dusty Strömgren; scattering bracket; T_d/IR
`peel`	peel-off images vs. the optically thin analytic flux (+0.017%); bin cubes
`uni_dda`	uniform grid: `'amr'` walk ≡ octree (bit-identical); DDA walk to rounding
`pdr`	FUV/PDR switches: carbon-electron shell, photoelectric T_e
`ext_field`	isotropic external field: interior $J = J_{\rm ext}$ to 0.2% (rec and sph)
`multi_src`	multiple point sources: superposition, spectrum resolution, one-file spectra
`peel_ext`	external-field peel: silhouette flux vs. $JA_{\rm proj}/d^2$, chord τ spectrum
`qmc`	quasi-random launch: Sobol generator vs. `scipy` + Owen-net balance; the fixed-state variance gate (cell-wise $\Gamma_{\rm HI}$ gain 29× at 2^{17}, 222× at 2^{20})
`hei_meta`	He I 2^3S metastable: OH18 analytic limits, module n_3 vs. the Python closed form, Φ_3 from the band tally, and the 10833 Å column/optical-depth diagnostic (conservation + hand integral)

A minimal thermal + metals + dust input (from `d_dusty`):

```
&parameters
 par%no_photons = 4.0e7          par%iseed = 1234
 par%grid_type = 'amr'           par%amr_file = 'amr_L6.fits'
 par%distance_unit = 'pc'        par%dust_density_law = 'global_dgr'
 par%nnu_ion = 32
 par%eion_min = 13.598           par%eion_max = 100.0
 par%tstar = 4.0e4               par%luminosity = 3.178e38
 par%xHI_init = 1.0e-3           par%abund_He = 0.1
 par%gas_niter = 60              par%gas_tol = 2.0e-3
 par%solve_te = .true.           par%atomic_dir = '../../data/atomic'
 par%use_metals = .true.
 par%ion_add_dust = .true.
 par%ion_dust_kext = '../../data/kext_albedo_WD_MW_3.1_60_D03.all_2003'
 par%cext_dust = 4.868e-22       par%DGR = 1.0
 par%out_file = 'run.h5'         par%file_format = 'hdf5'
/
```

5. Adding an ion

Adding an ion touches only data:

1. `tools/fitting/fit_cooling.py`: add the ion with a T_i ladder guessed from its `.elvlc` level energies; run; check the reported max error and the overlay figure.
2. `tools/fitting/fit_nlevel.py`: add the ion with the level count needed for its diagnostics; run.
3. `tools/fitting/make_element_data.py`: add the element to `ELEMENTS` (Z, stage count) and its thresholds; charge exchange entries where sources exist; run. The CHIANTI `.rrparams`/`.drparams` types 1–3 are handled (`RR`/`RR2`/`DR`/`DR2` lines).
4. Fortran: a `par%abund_<El>` field and one entry in the `species_setup` name/abundance lists (only for a NEW element; new stages of a known element need nothing).
5. Verify: extend `verify_nlevel_pyneb.py` with a diagnostic ratio where PyNeb has data; run a smoke test and check the new lines in `_lines.txt`.

Data caveats met so far: Ar II has no CHIANTI v11 level/collision data (stage tracked without lines); Fe III collision strengths differ from PyNeb's BB14 by up to ∼16% in 4658/4702 (known spread for iron); charge-exchange stage labels differ between the Huang (2023) and MOCASSIN transcriptions for Mg/Fe (the Huang labels are adopted; see `make_element_data.py`).

6. Practical notes

- **Bins**: 32 bins give ∼2% rate-integral discretization; use 64 for sub-percent work. The `add_fuv` extension carries its own bins below `eion_min`, so the ionizing resolution is unaffected.
- **FUV and the PDR**: without `add_fuv` there is no radiation beyond the I-front and the metals there recombine toward neutral; with it, FUV penetrates the I-front and photoionizes the metal stages whose thresholds lie below 13.6 eV (C I 11.3, S I 10.4, Si I 8.2, Fe I 7.9, Mg I 7.6 eV, . . .), producing the PDR-side C II and Mg II. In the FUV band the gas opacity is exactly this low-threshold metal photoionization (`ion_metal_abs`, on by default; H and He do not absorb below 13.6 eV), so the FUV is attenuated by that photoionization together with dust — only where those metals are already ionized does dust extinction alone remain. The metal stage fractions iterate with this opacity self-consistently; the corresponding metal photoheating and the grain photoelectric heating are not in the thermal balance, so PDR-zone temperatures are indicative only.
- **Convergence**: use `conv_crit = 'vol'` for production — the cell-max criterion stalls at the front-cell noise floor (and, with `add_fuv`, at no-heating cells where T_e pins to `te_min`). Both measures are printed each iteration. The explicit diffuse field smooths the front and converges far better.
- **Starts**: prefer an ionized initial state; a neutral start advances the front ∼one cell per iteration.

- **kext tables**: the default astrodust and DL07 tables cover the EUV directly (astrodust: 1.0×10^{-4}–$3.981 \times 10^{4}\,\mu$m; DL07: 6.2054×10^{-5}–$3.981 \times 10^{4}\,\mu$m). If `ion_dust_kext` names an explicit table, that table must cover the transported band; tables are sorted on read (the D03 file has an out-of-order seam).
- **Re-refinement**: one event per run; the old shared-memory windows are reclaimed only at exit.
- **SEDust**: `sed_workdir` must point at a SEDust `sed/` tree; the exact stochastic solve costs $\sim$0.05–0.1 s for each gas leaf (distributed over ranks).